

TAWAZON

COLLECTION
MANAGEMENT

Dashboard Hosting & Deployment Guide

Choosing where to host the Collection Dashboard in production —
with local document storage and a PostgreSQL database.

Prepared for the Tawazon Collection Dashboard · June 2026

1. Overview & Your Requirements

This guide compares hosting options for moving the Tawazon Collection Dashboard (a Django application) from local development to a live production environment.

Your stated plan

- **Uploaded documents** (case attachments) are stored on the server's **local disk** — not in the database or cloud object storage.
- **Text data** (clients, debtors, cases, payments, etc.) is stored in a **database**.
- The app already sends email via **Zoho SMTP** (for client OTP login) — this carries over unchanged.
- The public marketing website is hosted separately on Netlify and links to this dashboard's login page.

The two technical requirements this creates

- **Persistent file storage** — the host must keep files on disk permanently (many platforms wipe the disk on every redeploy).
- **A production database** — move from SQLite (development only) to **PostgreSQL**.

2. The Critical Gotcha: "Ephemeral" Disks

This single concept decides which host is right for you.

Most modern hosting platforms (Render, Railway, Heroku, etc.) run your app on an **ephemeral filesystem** — a temporary disk that is **completely erased every time the app restarts or redeploys**. Your database is safe (it's separate), but any file saved to a local `media/` folder would **disappear**.

What this means for you

Because your plan is to store client documents on local disk, you must choose a host that provides a **persistent disk**. Otherwise, uploaded documents would be lost on the next deployment. The options below are evaluated primarily on how well they handle this.

3. Hosting Options Compared

Criteria	PythonAnywhere ★	Railway	Render	Hetzner VPS
Approx. cost / month	~\$5 (flat)	~\$5–15 (usage)	~\$15+	~€4.5 / ~\$5
Local files persist?	Yes — natively	Only with a volume	Only with a disk	Yes — real disk
Database	Postgres / MySQL included	Managed Postgres	Managed Postgres	You install Postgres
Setup effort	Easy	Easy	Easy	Hard
Server maintenance	None (managed)	None (managed)	None (managed)	You manage it all
Latency to Oman	Moderate (EU/US)	Moderate (EU)	Moderate (EU)	Good (Germany)
Free-tier catch	Limited but no sleep	Usage-based billing	Sleeps; DB expires 90d	No free tier
Custom domain	Yes	Yes	Yes	Yes

4. Option Details

PythonAnywhere EASIEST FOR YOUR PLAN

Why it fits: Its filesystem is **persistent by default**, so local document uploads "just work" with no extra configuration — no volumes, no object storage. It is purpose-built for Django, so deployment is a guided web interface rather than server administration. Flat, predictable ~\$5/month.

Trade-offs: Servers are in the US/EU (moderate latency to Oman); less flexible than a full server; scaling options are limited. Perfectly adequate for a small internal dashboard.

Railway BEST MODERN PaaS

Why it fits: Excellent developer experience — deploy straight from a Git repository, managed PostgreSQL in a click, and no idle-sleep. Supports a **persistent volume** that you mount at the document folder so files survive redeploys.

Trade-offs: You must configure the volume (one extra step). Pricing is **usage-based**, so the monthly bill is less predictable than a flat fee. Attaching a volume restricts the app to a single instance (fine at this scale).

Render

Why it fits: Similar to Railway — managed Postgres, Git deploys, supports **persistent disks**.

Trade-offs: The **free web tier sleeps after 15 minutes** of inactivity (a client clicking "Login" would wait ~30s for it to wake), and the **free database expires after 90 days**. A non-sleeping setup costs ~\$15+/month. For a login portal, the sleeping behaviour is a real drawback — Railway is the better choice of the two.

Hetzner VPS CHEAPEST / FULL CONTROL

Why it fits: A real virtual server with a real disk. Documents live in `media/` permanently, Postgres runs on the same machine, and you have complete control. Lowest cost (~€4.5/mo) and best latency to Oman (German data centre).

Trade-offs: **You become the system administrator** — installing and securing Nginx, Gunicorn, PostgreSQL, SSL certificates, OS security updates, and backups. More setup and ongoing responsibility. (This can be largely scripted as a one-time job.)

5. The Database: SQLite → PostgreSQL

The dashboard currently uses **SQLite**, which is excellent for development but **not suitable for production**:

- It is a single file that handles concurrent users poorly.
- On managed platforms it would sit on the ephemeral disk and risk being wiped.
- It lacks the robustness, backups, and concurrency of a real database server.

The fix is **PostgreSQL**, which Django supports natively. On PythonAnywhere, Railway, and Render, a managed (and automatically backed-up) Postgres database is provided for you. On a VPS you install it yourself. The code change is small — point the app at the database connection string and add the Postgres driver.

6. Important: Backing Up Local Files

Storing documents on local disk means there is **no built-in redundancy**. If the server's disk fails, the files are gone. This is the main downside of local storage versus cloud object storage (which is replicated automatically).

Whichever host you choose, you must set up your **own backup** of the documents folder — for example, a nightly automated copy of `media/` to a second location (another disk, a cloud bucket, or an offsite backup). The text database is backed up by the managed platform, but the files are your responsibility.

7. Recommendation

There is no wrong choice here — it depends on which priority matters most to you:

- 1 **Want the easiest path where local files simply work?** → **PythonAnywhere** (~\$5/mo flat). Fewest moving parts; persistent disk by default; built for Django. *Recommended starting point.*
- 2 **Want a modern Git-based platform and don't mind configuring a volume?** → **Railway** (~\$5–15/mo). Great experience, managed Postgres, no sleeping.
- 3 **Want the lowest cost and full control, and are comfortable being the sysadmin?** → **Hetzner VPS** (~€4.5/mo). Cheapest long-term, best latency to Oman, most responsibility.

Suggested choice for Tawazon

For a small internal dashboard where you want documents on local disk with minimal fuss, **PythonAnywhere** offers the best balance of *easy + affordable + your file-storage plan working out of the box*. If you prefer a more modern platform, **Railway** is an equally solid choice that adds a managed database and Git-based deploys for a little more configuration and cost.

8. Pre-Deployment Checklist

Regardless of host, these changes prepare the codebase for production:

- Switch the database from SQLite to **PostgreSQL** (via a `DATABASE_URL` setting).
- Set `DEBUG = False` and a strong, secret `SECRET_KEY` from an environment variable.
- Restrict `ALLOWED_HOSTS` and `CSRF_TRUSTED_ORIGINS` to your real domain.
- Move secrets (Zoho email password, secret key) into **environment variables** — never hard-coded.
- Point `MEDIA_ROOT` at the **persistent disk** location and confirm uploads persist.
- Serve static files with **WhiteNoise** (or the platform's static handling).
- Run the app with a production server — **Gunicorn** — not the Django dev server.
- Add a generated `requirements.txt` pinning the project's dependencies.
- Set up automated **backups** for both the database and the `media/` documents folder.
- Add an **SSL certificate** (HTTPS) — automatic on managed hosts; Let's Encrypt on a VPS.